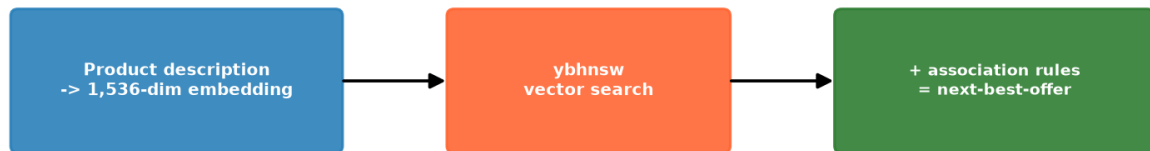


Vector Search for Real Recommendations



A real H&M retail dataset, a distributed HNSW index, and an honest look at the benchmark numbers

Figure 1: Figure 0: Vector Search for Real Recommendations

Welcome to this edition of yugabyteDB Developer’s Notebook (YDN). This month we answer the following question(s);

We keep seeing pgvector demos that do a single nearest-neighbor lookup against a toy dataset and call it done. We want to build an actual “next best offer” recommendation feature – real product data, real customer behavior, a real distributed HNSW index – and we want honest numbers, not a marketing benchmark. What does that actually look like on yugabyteDB ?

A fair ask, and this month’s article is built entirely around a real dataset (H&M’s public fashion-retail catalog and transaction history), a real embedding model run locally, yugabyteDB’s own distributed HNSW index type, and a real published benchmark’s actual recall/latency numbers – including the explicit caveat attached to those numbers, not just the headline. We also cover why a production recommendation feature is never just a vector lookup by itself, and show the real hybrid query (vector similarity plus association-rule mining) this project actually runs.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0 with the **vector** (pgvector-compatible) extension, Python 3 with **transformers/torch** for local embedding generation, and the **Alibaba-NLP/gte-Qwen2-1.5B-instruct** embedding model. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB’s managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

4.1 Terms and core concepts

4.1.1 Introduction: A Real Dataset, Not a Toy One

This month’s project uses H&M’s public “Personalized Fashion Recommendations” dataset (a real Kaggle competition dataset) – an **articles** table (product catalog: name, product type, colour, department, gar-

ment group, a free-text `detail_desc`), a `customers` table, and a `transactions` table of real purchase history. Each article gets a 1,536-dimensional embedding, computed from its own descriptive text.

Embeddings are generated locally, not through an external API call – a real, loadable model (Alibaba-NLP/gte-Qwen2-1.5B-instruct, a specific pinned revision) run directly via `transformers/torch`. Query-side encoding uses an explicit instruction-formatted prompt, matching how this particular embedding model was trained to be queried:

Example 4-1: The actual query-encoding template

```
query_text = (
    "Instruct: Given a product description, retrieve relevant retail "
    f"products\nQuery: {predicate}"
)
tokens = tokenizer([query_text], max_length=512, padding=True, truncation=True, return_tensors="pt")
with torch.inference_mode():
    hidden = model(**tokens).last_hidden_state
    embedding = functional.normalize(hidden[:, -1], p=2, dim=1)[0]  # last-token pooling, L2-normalized
```

[!NOTE] The instruction prefix (“Instruct: Given a product description, retrieve relevant retail products”) is not decorative – `gte-Qwen2-1.5B-instruct` was trained expecting this exact framing at query time, and last-token pooling (taking the final hidden state rather than averaging across tokens) is specific to this model family. Using a different embedding model without matching its own expected pooling and prompting convention is a common, easy way to quietly get worse retrieval quality without any error being raised.

4.1.2 Building the Index Before the Data Exists

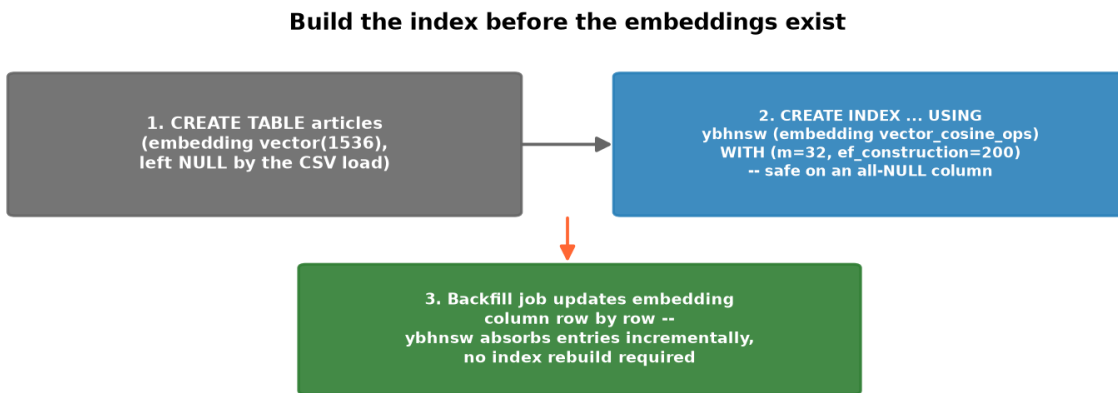


Figure 2: Figure 1: Build the index before the embeddings exist

Example 4-2: The real schema and index

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```

CREATE TABLE articles (
  article_id      text PRIMARY KEY,
  product_type_name  text,
  colour_group_name  text,
  detail_desc       text,
  -- Populated by a later embedding job:
  embedding        vector(1536),
  embedding_model   text,
  embedded_at       timestampz
);

-- Safe to create while embedding is entirely NULL:
CREATE INDEX NONCONCURRENTLY IF NOT EXISTS articles_embedding_cosine_idx
  ON articles USING ybhnsw (embedding vector_cosine_ops)
  WITH (m = 32, ef_construction = 200);

```

`ybhnsw` is yugabyteDB’s own distributed HNSW (Hierarchical Navigable Small World) vector index access method – the mechanism referenced in this project’s own supporting conference-talk abstract as “how yugabyteDB shards and distributes HNSW vector indexes across nodes.” `m` and `ef_construction` are standard HNSW construction parameters (graph connectivity and construction-time search breadth, respectively) – `m = 32`, `ef_construction = 200` here is a deliberately higher-recall-leaning choice than HNSW’s common defaults, appropriate for a recommendation feature where retrieval quality directly affects what a customer sees.

The schema comment in this project’s own DDL is worth calling out directly: *“It is safe to create this while embeddings are NULL. Later embedding updates add entries to the distributed HNSW index.”* This is a genuinely useful operational pattern for a real embedding rollout – the table and index can exist from day one, and a long-running backfill job can populate `embedding` incrementally afterward, with each `UPDATE` adding that row into the HNSW graph as it completes, rather than requiring the entire dataset to be embedded before the index can be built at all.

4.1.3 A Real Benchmark, Read Honestly

A separately run, real benchmark (not this project’s own H&M dataset, but the standard `glove-100-angular` ANN-Benchmarks corpus – approximately 1,183,514 vectors, 100 dimensions, a query set of 10,000 lookup vectors) measured yugabyteDB’s HNSW implementation (`YB-Hnswlib`) at several recall targets:

Recall	QPS	Approx. time per lookup
90.5%	1,066	0.94 ms
93.3%	872	1.15 ms
96.2%	645	1.55 ms
98.5%	403	2.48 ms
99.3%	297	3.36 ms
99.6%	237	4.21 ms

The clean headline – roughly 1,183,514 rows, 100 dimensions, a 1-4 ms nearest-neighbor lookup depending on desired recall – is real, but comes with an important, explicitly stated qualification worth repeating rather than dropping: this is benchmark **service time**, derived directly from measured queries-per-second (`lookup time 1,000 / QPS` milliseconds), not necessarily full end-to-end application latency. It may exclude connection setup, network latency, result serialization, concurrency effects under real load, and the cost of any additional filtering layered on top – exactly the kind of layering section 4.1.4 covers next.

[!NOTE] Recall and speed trade off directly against each other in HNSW, visible directly in the table above – QPS drops by more than 4x between the 90.5% and 99.6% recall targets. There

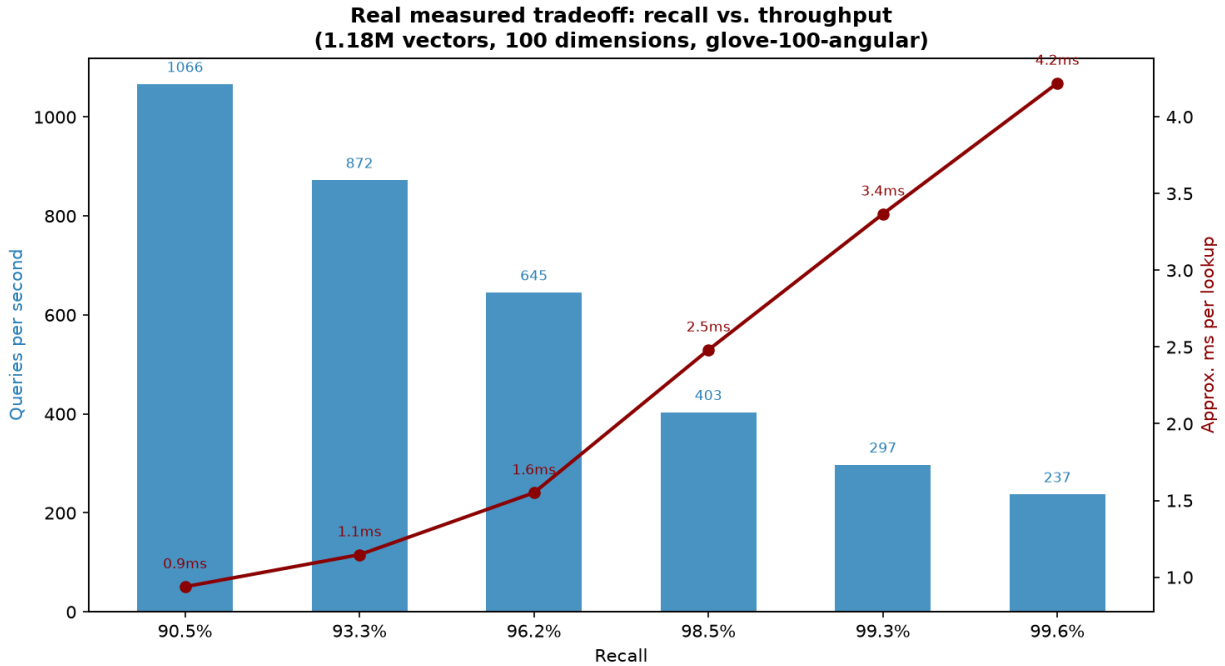


Figure 3: Figure 2: Real measured tradeoff – recall vs. throughput

is no single “correct” recall target; it’s a genuine product decision, weighing how much a missed near-neighbor actually costs (a slightly-less-relevant recommendation) against how much added latency a real request budget can absorb.

4.1.4 Vector Search Is the First Stage, Not the Whole Feature

A pure nearest-neighbor lookup answers “what’s semantically similar” – it does not by itself answer “what should we actually recommend,” which depends on real business context a vector alone can’t encode: is the item in stock, is it in the customer’s region, has this customer already rejected it, does buying it typically pair with something else. This project’s real recommendation query layers a second signal on top of the vector search: a `product_type_associations` table recording market-basket-style co-occurrence statistics (`pair_count`, `confidence`, `lift`) mined from real transaction history.

Example 4-3: The actual hybrid query – semantic similarity plus association rules

```
WITH relationships AS MATERIALIZED (
    SELECT offer_product_type, pair_count, confidence, lift
    FROM product_type_associations
    WHERE trigger_product_type = %s
    ORDER BY (ln(1 + pair_count) * ln(1 + lift) * confidence) DESC, pair_count DESC
    LIMIT 3
),
nearest AS MATERIALIZED (
    SELECT article_id, prod_name, product_type_name, colour_group_name,
           embedding <=> %s::vector AS distance
    FROM articles
    WHERE embedding IS NOT NULL
    ORDER BY embedding <=> %s::vector
    LIMIT %s
)
```

```

SELECT nearest.article_id, nearest.prod_name,
       nearest.product_type_name AS complementary_type,
       nearest.colour_group_name
FROM nearest
JOIN relationships ON relationships.offer_product_type = nearest.product_type_name;

```

The ranking expression behind `relationships - ln(1 + pair_count) * ln(1 + lift) * confidence` – deliberately dampens raw pair-count with a logarithm (so one extremely common product-type pairing doesn’t dominate purely on volume), while still rewarding both a strong lift (this pairing happens much more than chance alone would predict) and a strong confidence (given the trigger, this complementary type reliably follows). This is the concrete version of the “next best offer” pattern: **customer/context** -> **embedding** -> **vector search** retrieves candidates -> **business rules and association data** narrow and re-rank -> **final recommendation** – vector search supplies the *candidate generation* stage, not the final answer by itself.

4.1.5 What an Honest Recommendation Benchmark Actually Needs to Test

The 1-4 ms figures in section 4.1.3 are real, but they measure an isolated vector lookup against a fixed, static corpus – not a live recommendation feature. A benchmark that would actually validate a production “next best offer” feature needs to cover considerably more ground: realistic scale (1M, 10M, 100M vectors, not just one fixed size), realistic dimensionality (128-1,536, matching a real embedding model rather than a convenient benchmark corpus), **filtered** vector search (region, inventory, eligibility – combined with the similarity search, not run separately), continuous updates (offers expiring, customer preference vectors changing while queries are still running), concurrent load, p50/p95/p99 latency rather than only an average, Recall@K measured against an exact-search ground truth, and distributed behavior under real conditions – adding nodes, rebalancing, tolerating a node failure mid-query.

[!NOTE] The most compelling case for a distributed SQL database here isn’t “can it beat a specialized in-memory vector library on a raw nearest-neighbor benchmark” – it generally won’t, because a database is also supplying transactions, persistence, replication, and live relational data that a specialized vector library doesn’t have to. The stronger, more honest claim is that yugabyteDB can produce a high-quality recommendation using vector similarity *together with* live eligibility filters and transactional customer/offer data, with predictable latency, while the underlying data keeps changing and the cluster keeps running – which is what section 4.1.4’s hybrid query is actually demonstrating.

4.2 Complete the following

In this section, we build the real schema, create the HNSW index ahead of the data (as section 4.1.2 describes), and run both a pure semantic search and the hybrid recommendation query from section 4.1.4.

4.2.1 Prerequisites

A yugabyteDB cluster with the `vector` extension available, Python 3 with `psycopg2`, `torch`, and `transformers` installed, and enough local compute to run a 1.5B-parameter embedding model (a GPU helps but is not required for a small demonstration corpus).

4.2.2 Step 1 – Create the schema and index, before loading any embeddings

Example 4-4: Reproducing section 4.1.2’s create-before-populate pattern

```

CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE articles (
  article_id text PRIMARY KEY,
  detail_desc text,
  embedding vector(1536)

```

```
);
CREATE INDEX NONCONCURRENTLY ON articles
  USING ybhns (embedding vector_cosine_ops) WITH (m = 32, ef_construction = 200);
```

Load a small sample of real or representative product rows with `embedding` left NULL, confirming the `CREATE INDEX` above succeeds against the empty column exactly as section 4.1.2 describes.

4.2.3 Step 2 – Backfill embeddings and confirm the index picks them up

Run an embedding job (using Example 4-1’s model and prompting convention, or any consistent embedding model of your choice) that updates `embedding` for each row, then confirm a semantic query against the now-populated table uses the `ybhns` index (via `EXPLAIN`) rather than a sequential scan.

4.2.4 Step 3 – Run a pure semantic query, then the hybrid version

Example 4-5: The semantic-only query behind this project’s search endpoint

```
SELECT article_id, prod_name, product_type_name,
       round((embedding <=> %s::vector)::numeric, 6) AS distance
FROM articles
WHERE embedding IS NOT NULL
ORDER BY embedding <=> %s::vector
LIMIT 10;
```

Run this against a query embedding, then run Example 4-3’s full hybrid version against the same query, using a small `product_type_associations` table you populate with a handful of realistic (`trigger_product_type`, `offer_product_type`, `pair_count`, `confidence`, `lift`) rows. Confirm the hybrid query’s results are additionally filtered/re-ranked by the association data, rather than being identical to the pure semantic result – demonstrating directly that the two stages (candidate generation, then business-logic re-ranking) genuinely compose, rather than one making the other redundant.

Section summary In this section, we built a real vector schema using yugabyteDB’s `ybhns` distributed index type, confirmed the create-before-populate operational pattern this project relies on for a real embedding rollout, and ran both a pure semantic-similarity query and the full hybrid recommendation query that combines it with association-rule data – confirming directly that the two stages produce genuinely different, complementary results rather than the second being redundant with the first.

4.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- A real recommendation-feature dataset (H&M’s public fashion-retail catalog and transaction history) and a real, locally-run 1,536-dimensional embedding model, including the specific instruction-prompting and pooling convention that model expects at query time.
- yugabyteDB’s `ybhns` distributed HNSW vector index type, its construction parameters (`m`, `ef_construction`), and a genuinely useful operational pattern this project relies on directly: creating the index before the embedding column is populated, then backfilling incrementally.
- A real, published benchmark’s actual recall-versus-throughput numbers on a 1.18-million-vector corpus, read together with the explicit caveat the benchmark itself states – service time, not necessarily full end-to-end application latency.
- Why vector search is a candidate-generation stage, not a complete recommendation feature by itself, demonstrated with this project’s own real hybrid SQL query combining cosine similarity with market-basket-style association-rule data (`pair_count`, `confidence`, `lift`).

- What an honest, production-representative recommendation benchmark actually needs to cover – filtered search, continuous updates, concurrent load, tail latency, and distributed failure tolerance – beyond a single isolated nearest-neighbor lookup number.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely's very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-02%20-%20Vector%20Search%20for%20Recommen%20Vector%20search%20for%20recommendations.pdf>